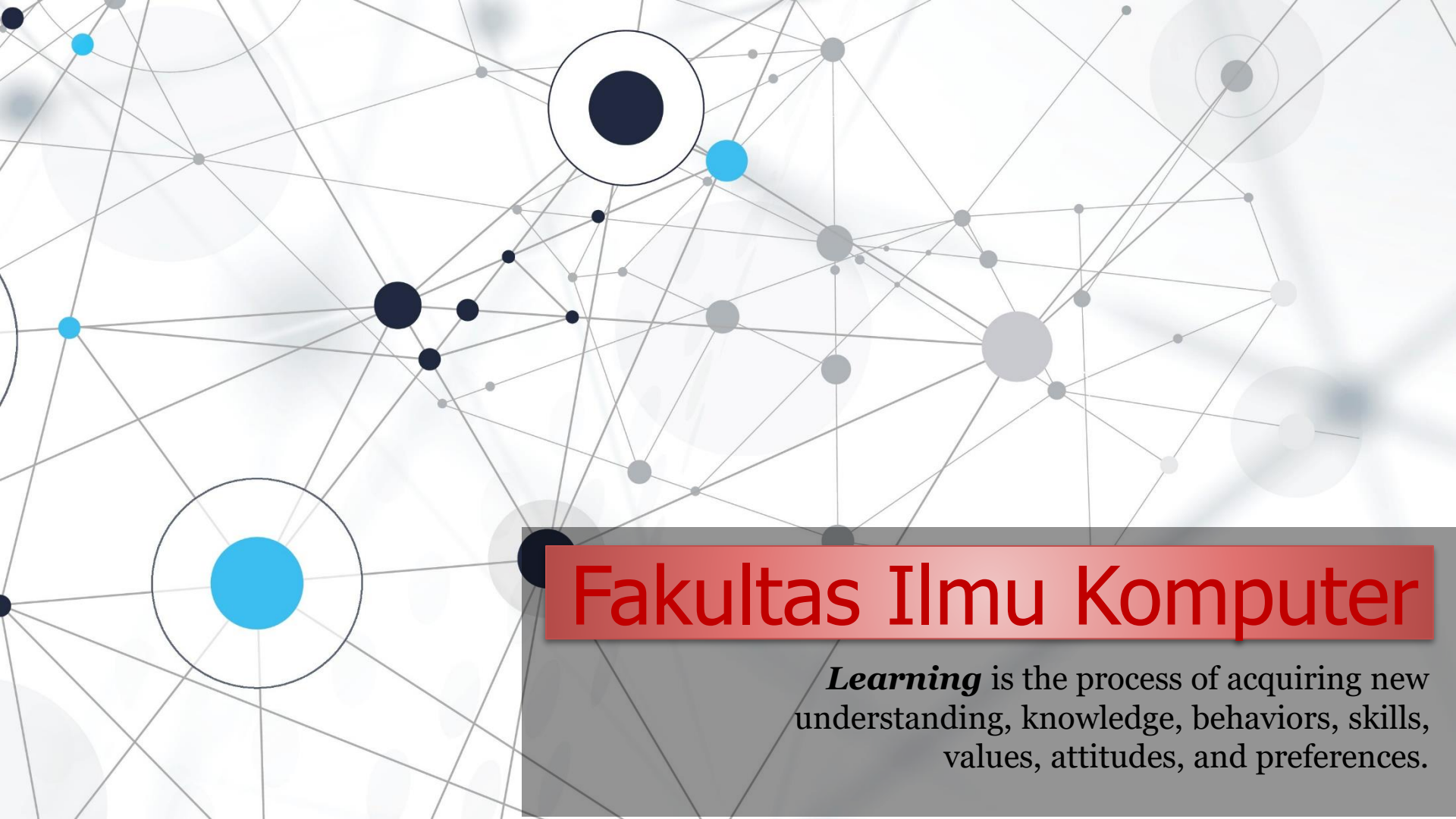


A complex network diagram with numerous nodes of varying sizes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger concentric circles. The background is light grey with faint circular patterns.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

DevOps Lifecycle

Continuous Integration (CI)



```
/var/log/auth.log.1:Sep 5 08:46:35 ISC2025 sshd[1594]: Connection closed by invalid user user 89.0.111.227 port 60547 [preauth]
/var/log/auth.log.1:Sep 5 08:46:54 ISC2025 sshd[1599]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:46:54 ISC2025 sshd[1599]: Connection closed by invalid user root 89.0.111.227 port 64871 [preauth]
/var/log/auth.log.1:Sep 5 08:47:13 ISC2025 sshd[1601]: Invalid user lixiaoke from 89.0.111.227 port 65022
/var/log/auth.log.1:Sep 5 08:47:13 ISC2025 sshd[1601]: Connection closed by invalid user lixiaoke 89.0.111.227 port 65022 [preauth]
/var/log/auth.log.1:Sep 5 08:47:32 ISC2025 sshd[1603]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:47:32 ISC2025 sshd[1603]: Connection closed by invalid user root 89.0.111.227 port 63643 [preauth]
/var/log/auth.log.1:Sep 5 08:47:52 ISC2025 sshd[1608]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:47:52 ISC2025 sshd[1608]: Connection closed by invalid user root 89.0.111.227 port 63631 [preauth]
/var/log/auth.log.1:Sep 5 08:48:11 ISC2025 sshd[1610]: Invalid user dmdba from 89.0.111.227 port 64404
/var/log/auth.log.1:Sep 5 08:48:11 ISC2025 sshd[1610]: Connection closed by invalid user dmdba 89.0.111.227 port 64404 [preauth]
/var/log/auth.log.1:Sep 5 08:48:30 ISC2025 sshd[1613]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:48:30 ISC2025 sshd[1613]: Connection closed by invalid user root 89.0.111.227 port 59373 [preauth]
/var/log/auth.log.1:Sep 5 08:48:51 ISC2025 sshd[1618]: User news from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:48:51 ISC2025 sshd[1618]: Connection closed by invalid user news 89.0.111.227 port 63701 [preauth]
/var/log/auth.log.1:Sep 5 08:49:10 ISC2025 sshd[1620]: Invalid user git from 89.0.111.227 port 64060
/var/log/auth.log.1:Sep 5 08:49:10 ISC2025 sshd[1620]: Connection closed by invalid user git 89.0.111.227 port 64060 [preauth]
/var/log/auth.log.1:Sep 5 08:49:29 ISC2025 sshd[1622]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:49:30 ISC2025 sshd[1622]: Connection closed by invalid user root 89.0.111.227 port 59397 [preauth]
/var/log/auth.log.1:Sep 5 08:49:49 ISC2025 sshd[1739]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:49:49 ISC2025 sshd[1739]: Connection closed by invalid user root 89.0.111.227 port 65154 [preauth]
/var/log/auth.log.1:Sep 5 08:50:09 ISC2025 sshd[1741]: Invalid user jenkins from 89.0.111.227 port 63977
/var/log/auth.log.1:Sep 5 08:50:09 ISC2025 sshd[1741]: Connection closed by invalid user jenkins 89.0.111.227 port 63977 [preauth]
/var/log/auth.log.1:Sep 5 08:50:27 ISC2025 sshd[1756]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:50:28 ISC2025 sshd[1756]: Connection closed by invalid user root 89.0.111.227 port 61509 [preauth]
/var/log/auth.log.1:Sep 5 08:50:46 ISC2025 sshd[1763]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:50:47 ISC2025 sshd[1763]: Connection closed by invalid user root 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:51:05 ISC2025 sshd[1787]: Invalid user stl from 89.0.111.227 port 59964
/var/log/auth.log.1:Sep 5 08:51:05 ISC2025 sshd[1787]: Connection closed by invalid user stl 89.0.111.227 port 59964 [preauth]
/var/log/auth.log.1:Sep 5 08:51:24 ISC2025 sshd[1791]: Invalid user db2 from 89.0.111.227 port 59687
/var/log/auth.log.1:Sep 5 08:51:24 ISC2025 sshd[1791]: Connection closed by invalid user db2 89.0.111.227 port 59687 [preauth]
/var/log/auth.log.1:Sep 5 08:51:42 ISC2025 sshd[1796]: User root from 89.0.111.227 not allowed because not listed in AllowUsers
/var/log/auth.log.1:Sep 5 08:51:43 ISC2025 sshd[1796]: Connection closed by invalid user root 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:52:01 ISC2025 sshd[1798]: Invalid user root from 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:52:02 ISC2025 sshd[1798]: Connection closed by invalid user root 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:52:20 ISC2025 sshd[1800]: Invalid user root from 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:52:20 ISC2025 sshd[1800]: Connection closed by invalid user root 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:52:38 ISC2025 sshd[1829]: Invalid user root from 89.0.111.227 port 64266 [preauth]
/var/log/auth.log.1:Sep 5 08:52:38 ISC2025 sshd[1829]: Connection closed by invalid user root 89.0.111.227 port 64266 [preauth]
```

status for the jail: sshd

- Filter

|- Currently failed: 1

|- Total failed: 1750

|- File list: /var/log/auth.log

- Actions

|- Currently banned: 9

|- Total banned: 10

|- Banned IP list: 89.0.111.227 222.241.248.136 61.177.238.130 2.54.47.72 14.18.190.138 171.222.186.114

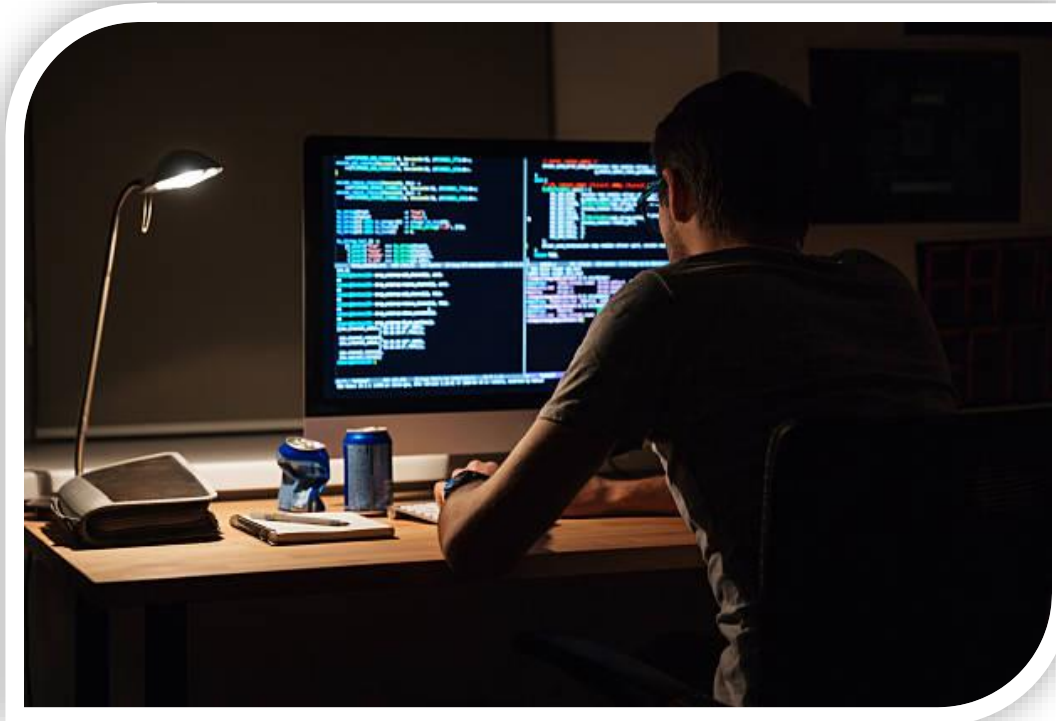
89.0.111.227

This IP has been flagged as abusive — see [AbuseIPDB](#) for details.

IP Geolocation

City	Köln
State	North Rhine-Westphalia
Country	 Germany
Postal	50667
Local time	09:14 AM, Tuesday, September 09, 2025
Timezone	Europe/Berlin
Coordinates	50.9333,6.9500



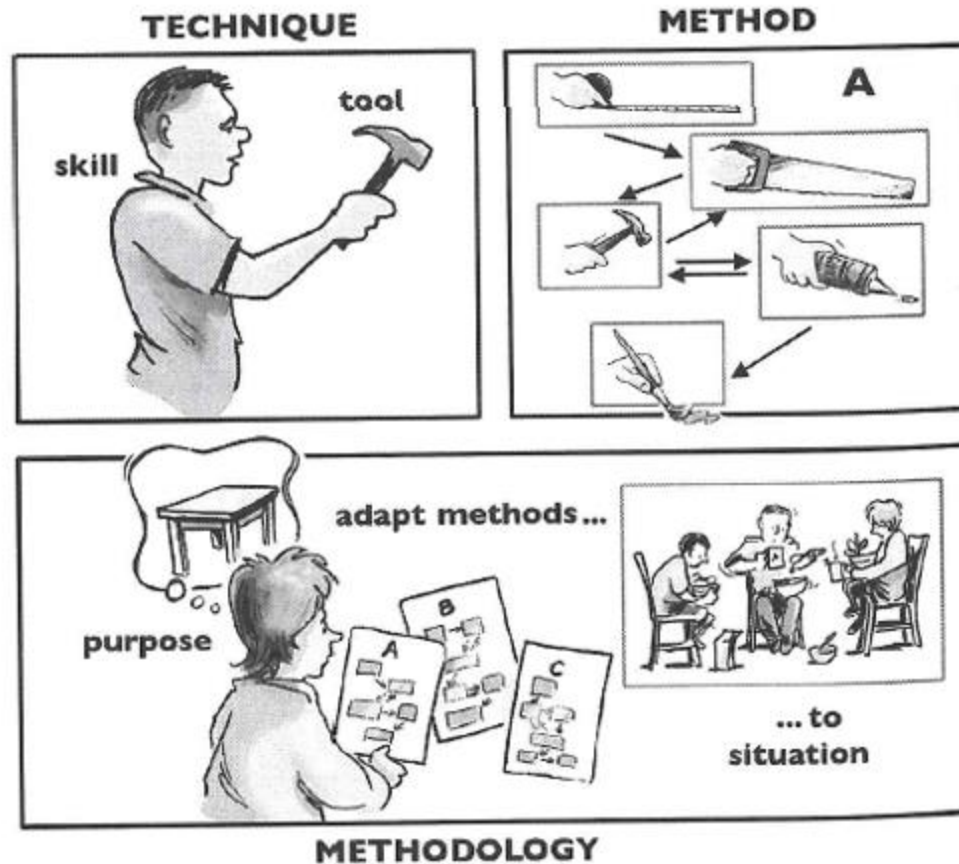


Jika kalian adalah seorang *programmer*, seberapa pahamkah anda terhadap semua tahapan di dalam SDLC? Apa perbedaan software developer dan software engineer?



Jika DevOps bertujuan untuk mempercepat siklus pengembangan dan mempercepat waktu pengiriman code dari local ke server side, maka apakah DevOps adalah sebuah teknologi, sebuah metode atau metodologi?

Method adalah cara atau prosedur spesifik.

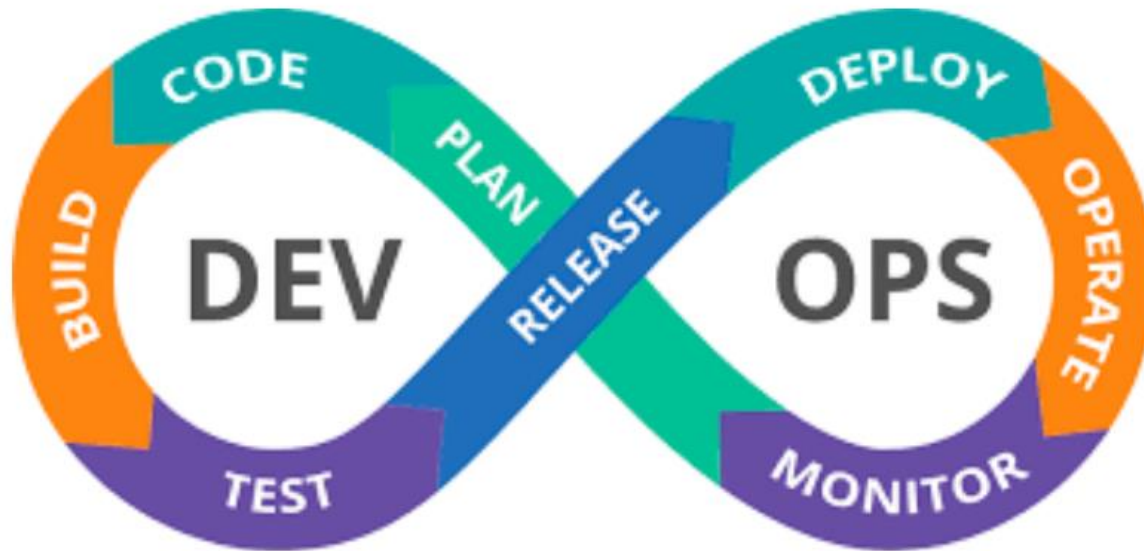


Metodologi adalah studi tentang method/metode. **DevOps** adalah metodologi yang menggabungkan praktik/skill, alat/tenologi dan pendekatan yang mencakup analisis dan evaluasi berbagai method/metode.

The slide features a white background with green leaves framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub, with some showing signs of aging or damage. The text is centered in a black serif font.

1 - Understanding DevOps Lifecycle

DevOps Methodology



What are the **CORE COMPONENTS** of DevOps?
What are the phases/terms followed in DevOps Methodology?

DevOps Methodology vs SDLC

No	Stage	Definition
1	Plan	Requirements are gathered from stakeholders and customers, then used to build a product roadmap that will guide future development. Popular tools like Jira or Azure DevOps can be used to track Epics, Features, and User Stories that will help in code development.
2	Code	The system/application is coded according to the business requirements.
3	Build	Once a developer has completed their task, they submit their code to a shared code repository. Usually, developers submit pull requests to merge their new code with the shared code.
4	Test	After a successful build, the application is automatically deployed to a staging environment for functional testing by quality engineers or business users. Many companies have their own testing process that is customized to what works best for the company.
5	Integrate	At this stage, code from various developers can be merged into a single branch. By this time, any code changes have passed a series of manual and automated tests.
6	Deploy	At this stage, the code is implemented in a production environment for use by end-users.
7	Operate	Based on the hosting service configuration, the environment can automatically scale the load to handle spikes and dips in the number of active users. Infrastructure as Code (IaC) is very useful at this stage.
8	Monitor	System/application performance is monitored. All this information is then fed back to the Product Manager and development team to close the loop in the process.

The DevOps Methodology

DevOps adoption is growing as development teams seek to become more agile. However, the underlying methodology requires organizations to instill the proper cultural and technical aspects in order to achieve success. The graphic below outlines what this entails.

Components of the DevOps Methodology

```
graph TD; Root[Components of the DevOps Methodology] --> Culture[Culture of DevOps Practices]; Root --> Technical[Technical DevOps Practices]; Culture --> Collaborating[Collaborating]; Culture --> Responsibility[Sharing responsibility]; Culture --> Focus[Making customer needs the primary focus]; Culture --> Automating[Automating tasks]; Culture --> Failure[Embracing failure]; Culture --> Autonomy[Providing team autonomy]; Technical --> CI[Continuous integration]; Technical --> CDD[Continuous delivery and deployment]; Technical --> CT[Continuous testing]; Technical --> MVP[Minimum viable product];
```

Culture of DevOps Practices

Collaborating

Sharing responsibility

Making customer needs the primary focus

Automating tasks

Embracing failure

Providing team autonomy

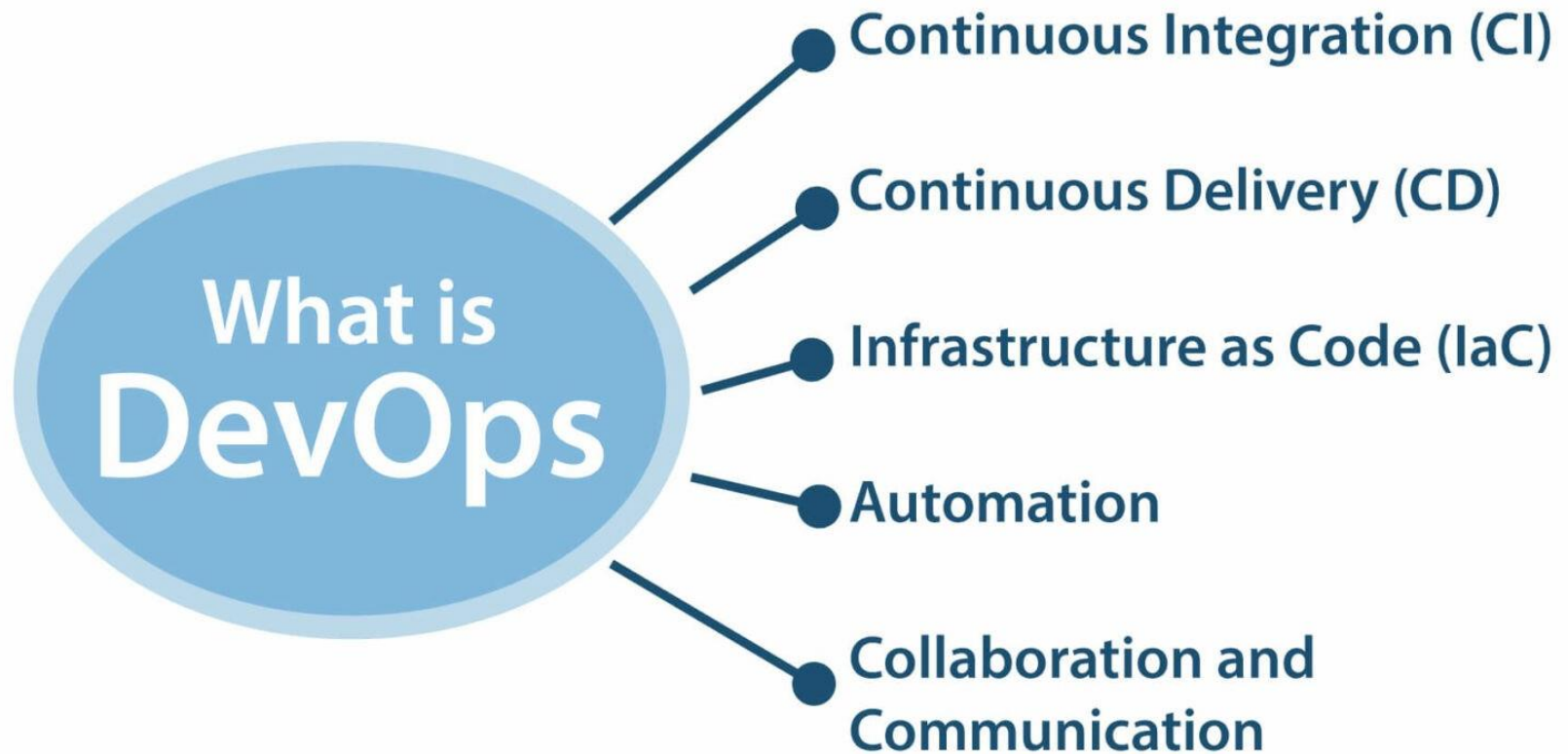
Technical DevOps Practices

Continuous integration

Continuous delivery and deployment

Continuous testing

Minimum viable product



DevOps merupakan studi tentang method dimana kita akan menggunakan berbagai metode untuk meningkatkan efisiensi dan kolaborasi antara Dev team (*pengembangan*) dan Ops team (*operasi*). Beberapa method/metode diantaranya yaitu Continuous Integration (CI) dan Continuous Deployment (CD) untuk otomatisasi kode, Infrastructure as Code (IaC) untuk pengelolaan infrastruktur otomatis, serta Configuration Management, Monitoring dan Logging untuk menjaga stabilitas dan keamanan. Penggunaan *containerization*, *microservices*, *version control*, *automated testing*, dan *continuous feedback* memungkinkan pengembangan cepat, pengujian otomatis, dan penyebaran aplikasi yang konsisten.

Key Components of DevOps



[1] Continuous Integration (CI)

Proses menggabungkan perubahan kode secara otomatis ke dalam ***shared repository (repositori Bersama)*** secara teratur, memungkinkan tim untuk secara cepat dan efisien mengidentifikasi dan memperbaiki konflik atau masalah integrasi.

Key Components of DevOps



[2] Continuous Testing (CT)

Proses otomatisasi *testing software* secara berkelanjutan untuk memastikan bahwa setiap perubahan *code* tidak memengaruhi kualitas atau kinerja aplikasi, sehingga mengurangi **risiko bug**.

Key Components of DevOps



[3] Continuous Development (CD)

Praktek pengembangan software (*perangkat lunak*) yang berfokus pada pengiriman perubahan atau fitur baru secara berkelanjutan (*continuous*), sehingga mengurangi waktu antara pengembangan hingga produksi.

Key Components of DevOps



[4] Continuous Deployment (CD)

Praktek mengotomatisasi *software release* (rilis) ke lingkungan produksi setelah melewati semua tahap testing dan persyaratan pengiriman, memungkinkan pengiriman perubahan *software* yang lebih cepat dan lebih andal.

Key Components of DevOps



[5] Continuous Monitoring (CM)

Komponen ini adalah salah satu komponen penting dalam DevOps yang berfokus pada pemantauan berkelanjutan untuk memastikan *performance*, *security* dan *application quality* dan infrastruktur yang sudah berjalan di produksi.

Key Components of DevOps



[6] Continuous Feedback (CF)

Pengumpulan dan analisis feedback (*umpan balik*) dari pemangku kepentingan, *users* atau proses otomatisasi untuk membantu tim pengembangan dalam meningkatkan produk dan proses pengembangan.

Chapter #1
DevOps Culture and Practices

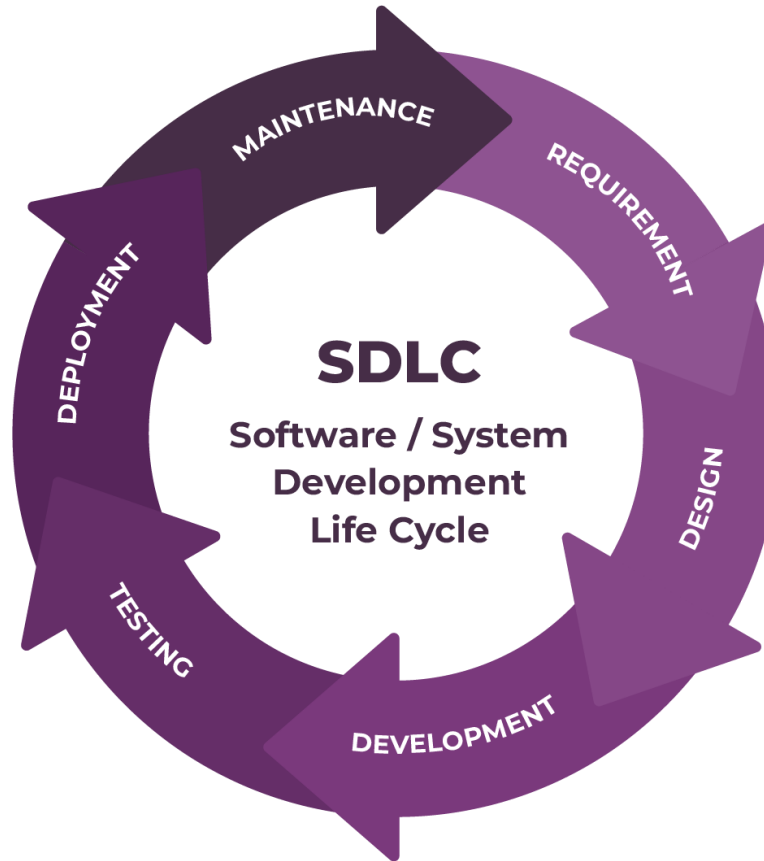
2 - DevOps *vs* SDLC



Getting Started *with* DevOps

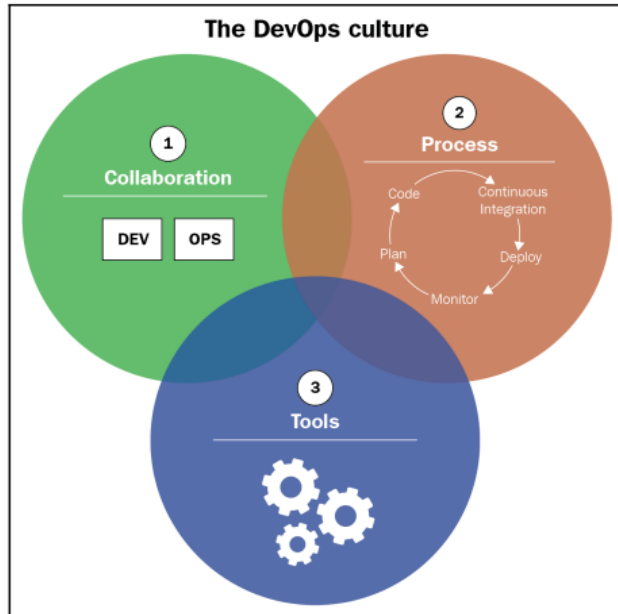
- ❖ The term DevOps was introduced in **2007-2009** by Patrick Debois, Gene Kim, and John Willis, and it represents the combination of **Development (Dev)** and **Operations (Ops)**.
- ❖ It has given rise to a movement that advocates **bringing developers and operations together within teams**.
- ❖ This is to be able to deliver added **business value** to users more **quickly and hence be more competitive in the market**.
- ❖ **DevOps culture** is a set of practices that reduce the barriers between
 - 1) **Developers** (*who want to innovate and deliver faster, on the one side*) and,
 - 2) **Operations** (*who want to guarantee the stability of production systems and the quality of the system changes they make*).
- ❖ DevOps culture is also the extension of **agile processes** (scrum, XP, and so on), which make it possible **to reduce delivery times** and already involve developers and business teams, but are often hindered because of the non-inclusion of Ops in the same teams.
- ❖ The communication and this link between Dev and Ops does, therefore, allow a better follow-up of end-to-end production deployments and more frequent **deployments of a better quality, saving money for the company**.

DevOps vs SDLC



Development (Dev) and **Operations (Ops)** processes are to be able (*aim to enhance the ability*) to deliver added business value to users more quickly and hence be more competitive in the market.

DevOps Cultures



The **benefits of establishing a DevOps culture within an enterprise** are as follows:

- 1) **Better collaboration and communication in teams**, which has a human and social impact within the company.
- 2) **Shorter lead times to production**, resulting in better performance and end user satisfaction.
- 3) **Reduced infrastructure costs** with Infrastructure as Code (IaC) .
- 4) **Significant time saved with iterative cycles that reduce application errors and automation tools that reduce manual tasks**, so teams focus more on developing new functionalities with added business value.

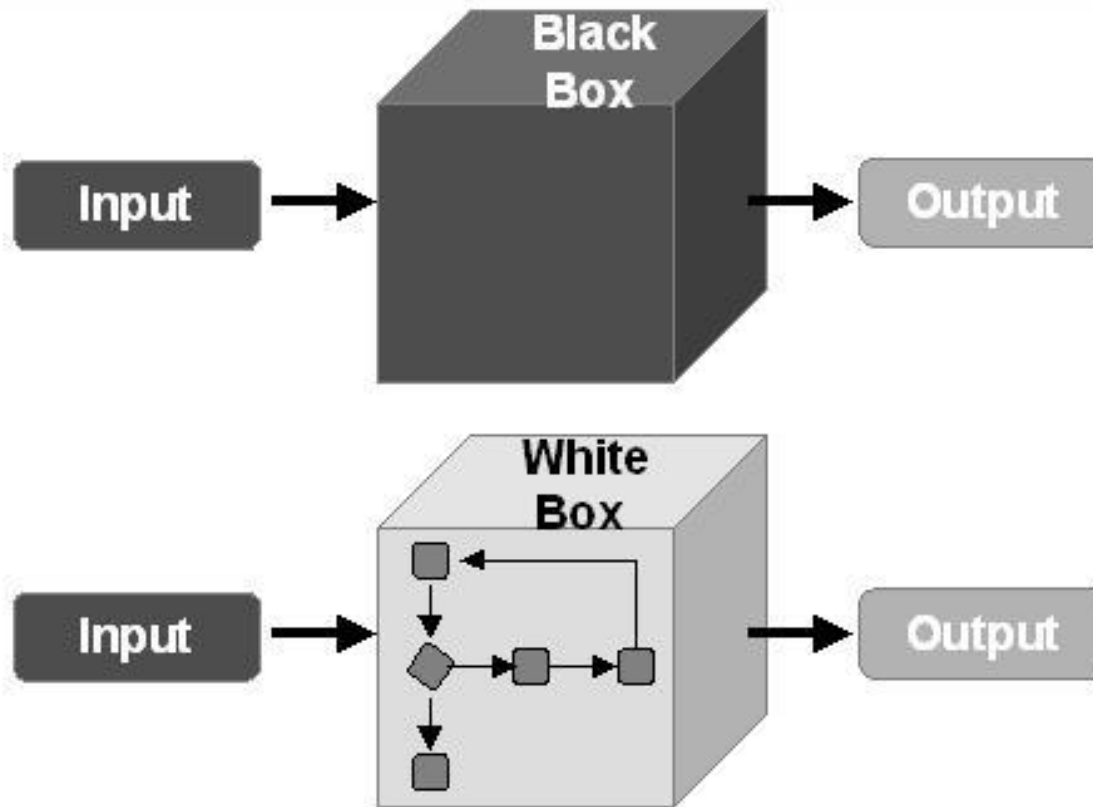
"DevOps is the union of people, process, and products to enable continuous delivery of value to our end users."

Key Elements DevOps

To facilitate this collaboration and improve communication between Dev and Ops, there are several key elements in the processes to be put in place, as in the following examples:

- 1) More frequent application deployments with **integration** and **continuous delivery** (called CI/CD).
- 2) The implementation and automation of unitary and integration tests, with a process focused on **Behavior-Driven Design (BDD)** or **Test-Driven Design (TDD)**.
- 3) The implementation of a means of collecting feedback from users.
- 4) **Monitoring applications and infrastructure.**

BDD *vs* TDD



Behavior-Driven Design (BDD) lebih fokus pada pengujian tingkat tinggi yang menilai perilaku dan fungsionalitas keseluruhan software/sistem dari perspektif end-users. Sedangkan, **Test-Driven Design (TDD)** lebih terkait dengan pengujian unit yang mendalam pada tingkat kode.



Mengapa keamanan di bagian back-end website sangat penting? Lebih khusus pada bagian memonitoring applications dan juga infrastructure?

The purpose of the "**hardening**" process on an Ubuntu 22.04 server is to improve system security by reducing potential security risks and closing loopholes that can be exploited by attackers. This includes measures to lock down the system, mitigate potential attacks, and minimize opportunities for unauthorized access.

- 1) Website **FRAMEWORK** with more security config approach.
- 2) MySQL root sudah dibuat password yg complex (sudo mysql_secure_installation).
- 3) MySQL user tanpa "default user name".
- 4) Firewall status actived (80, 443 allowed). Menggunakan UFW (Uncomplicated Firewall) atau iptables untuk mengkonfigurasi aturan firewall yang sesuai untuk server, dan hanya membuka port yang diperlukan.
- 5) SSH *port* sudah tidak menggunakan yang default (port 22).
- 6) User *root* password changed.
- 7) SSH *root* access disabled (PermitRootLogin no).
- 8) Sudah menggunakan public dan private RSA key untuk otentikasi SSH. Mencegah serangan **BRUTE FORCE** pada layanan yang memerlukan otentikasi, seperti SSH, dengan mengimplementasikan mekanisme pembatasan login dan penggunaan kunci SSH.
- 9) Http to HTTPS connection.

Chapter #1
DevOps Culture and Practices

3 - Continuous Integration (CI)



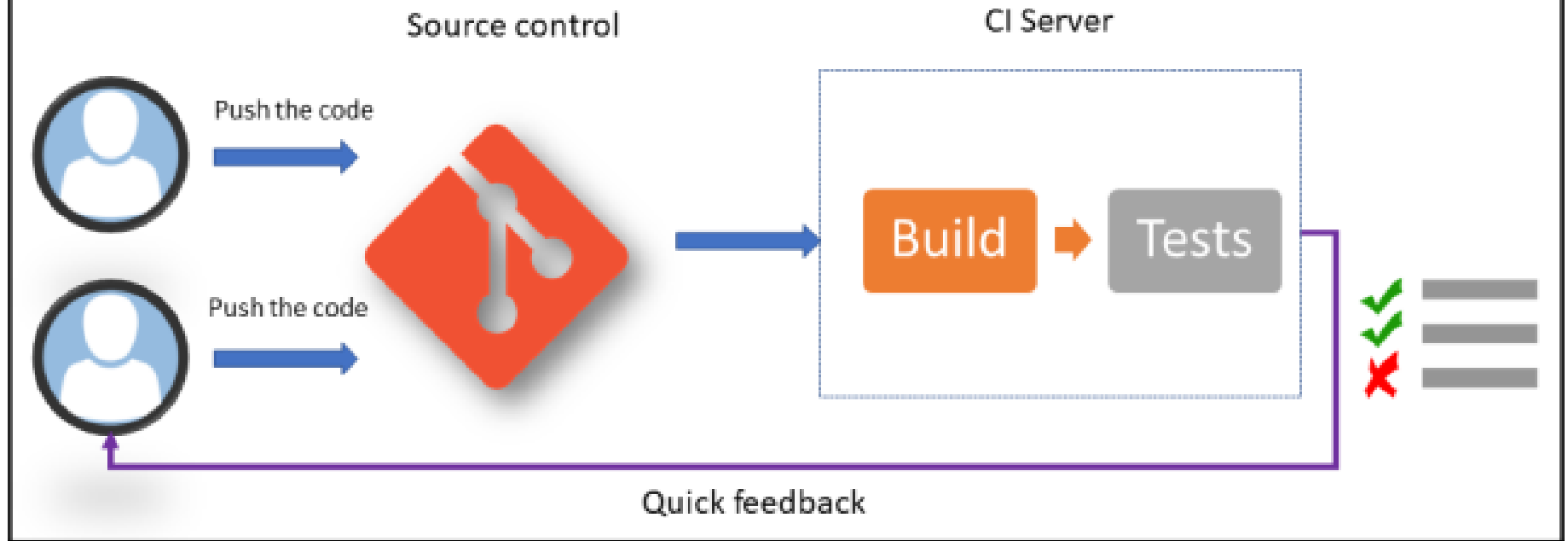
Continuous Integration (CI)

In the following definition given by **Martin Fowler**, there are three key things mentioned, members of a team, integrate, and as quickly as possible:

"Continuous Integration is a software development practice where members of a team integrate their work frequently... Each integration is verified by an automated build (including test) to detect integration errors as quickly as possible."

- ❖ With the spirit of collaboration and communication, because the execution of CI impacts all members in terms of work methodology and therefore collaboration.
- ❖ CI requires the implementation of processes (*branch, commit, pull request, code review, and so on*) with automation that is done with tools adapted to the whole team (**Git, Jenkins**, Azure DevOps, and so on).
- ❖ Finally, CI must run quickly to collect feedback on code integration as soon as possible and hence be able to deliver new features more quickly to users.

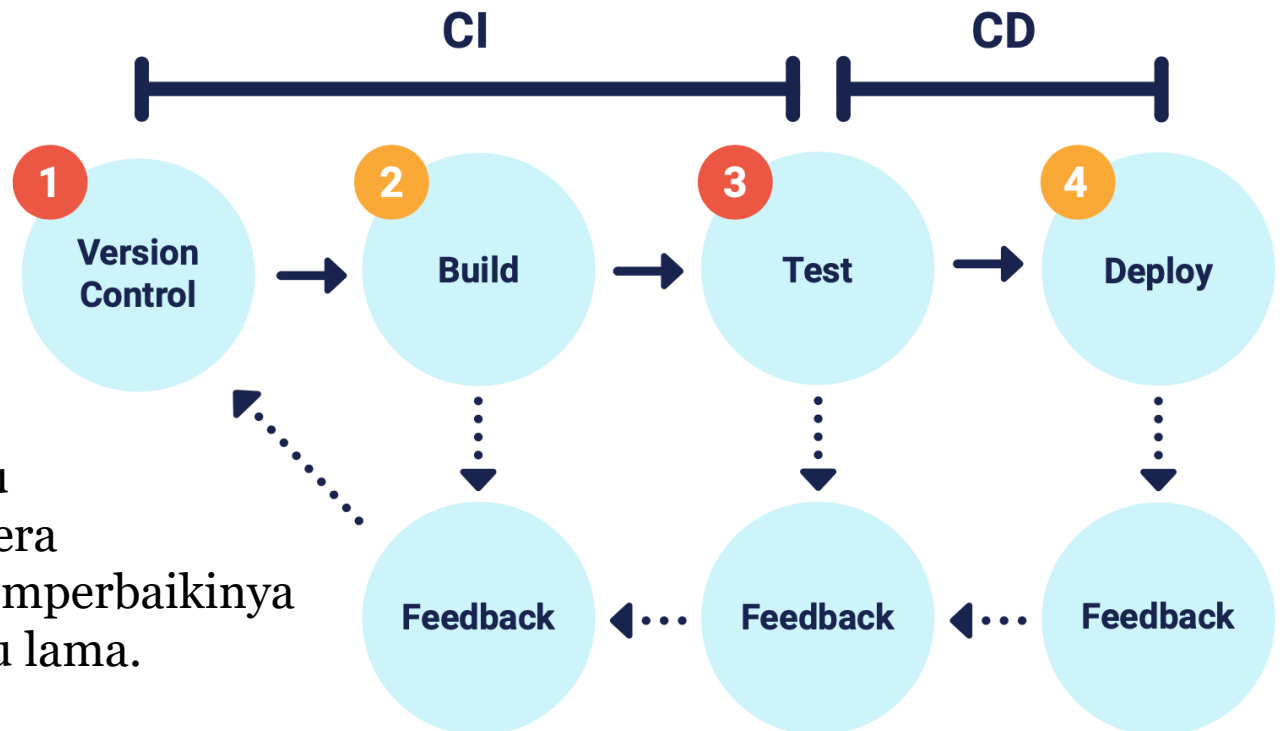
The continuous integration (CI) pipeline



- ❖ **This CI process must be optimized as soon as possible so that it can run fast and developers can have quick feedback on the integration of their code.**
- ❖ For example, code that is archived and does not compile or **whose test execution fails can impact and block the entire team.**
- ❖ This diagram shows the cyclical steps of **continuous integration that include the code being pushed into the Source Code Manager (SCM)** by the *team members* and the execution of the build and test by the **CI server**, and the purpose of this **fast process is to provide rapid feedback to members.**

Customer Feedback & Quick Response

*We are a team of operations engineers
doing software development - we are
DevOps!*



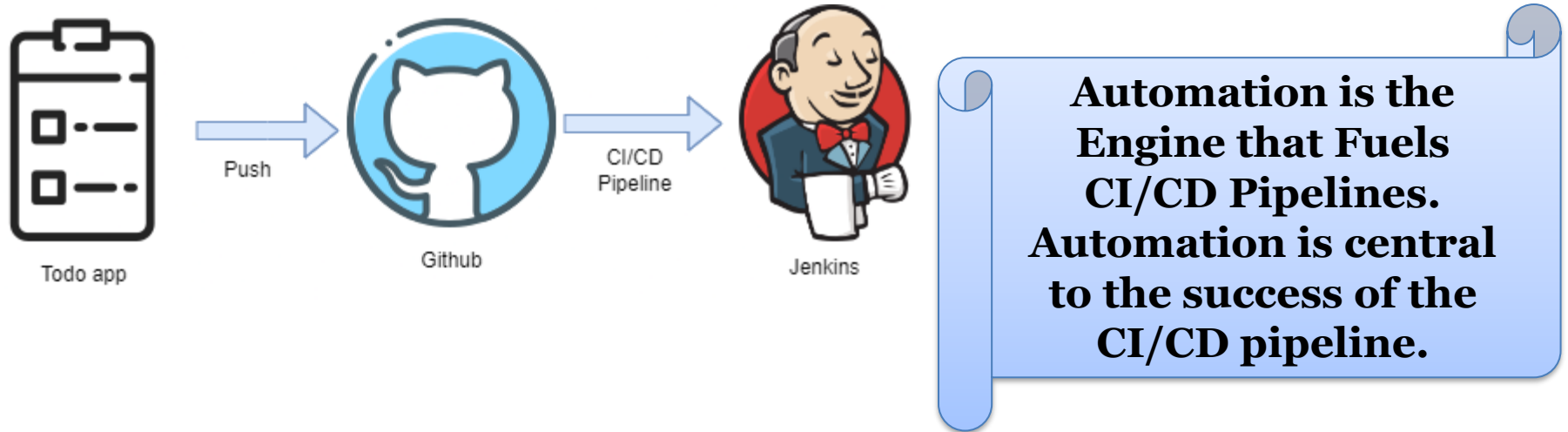
Jika ada kesalahan atau masalah, tim dapat segera mengetahuinya dan memperbaikinya tanpa menunggu terlalu lama.

Bad Practices == Failure Tests

- 1) **Bad practices of CI can result in the failure of tests in the CI**, deactivating the test execution, taking as arguments: it is not serious, it is necessary to deliver quickly, or the code that compiles it is essential.
- 2) This practice can have **serious consequences** when the **errors detected by the tests** are revealed in production.
- 3) The **time saved during CI will be lost on fixing errors** with hotfixes and redeploying them quickly with stress.

This process is the opposite of DevOps culture with ***poor application quality*** for end users and no real feedback, and, instead of developing new features, **we spend time correcting errors.**

Fuels CI/CD Pipelines



- ❖ Automation allows Developers to commit a change and see it in a deployable state within minutes. When developers introduce new commits, the system automatically generates a build and runs it through an automated testing.
- ❖ If it passes, the build moves on to the application servers and eventually gets released into the end-users.
- ❖ Every step of the way, admins are kept in the loop and notified if the build fails a test and it's all made possible by the **POWER OF AUTOMATION**.

Elestio Platform Managed Services

Elestio is fully managed DevOps platform to deploy our code and open-source software. Deploy otomatis dari GitHub/GitLab/Bitbucket. Elestio platform mendukung berbagai teknologi, bisa digunakan untuk **Next.js**, **Node.js**, Laravel, WordPress, **Django**, PostgreSQL, MySQL, dan banyak lagi.

elestio

PROJECT: default-project

Services

CI/CD

Volumes

Load Balancer

Domains

Members

Billing

Project Setting

Audit Trail

Account

Support Tickets

Documentation

Create CI/CD pipeline

1 Source ————— 2 Target

1. Deployment Method

Select the deployment method of your Service between Github, Gitlab and Docker.
When using the Github/Gitlab deployment method, each time a change is pushed to your repository, a new deployment of your service will be triggered.
To learn more about CI/CD deployment, see [this page](#).

Github
GitHub, Inc. is a provider of Internet hosting for software development and version control using Git. It offers the distributed version control and source code...

Gitlab
GitLab Inc. is the open-core company that provides GitLab, the DevOps software that combines the ability to develop, secure, and operate software in a single...

Docker Compose
Compose is a container orchestration tool that allows you to define and run multi-container Docker applications.

2. Git repository

Import Git Repository Clone Template

Import Git Repository

Git Account wass08 **Git Scope** wass08

Search repo



Exercise *for* Students

Questions

- 1) What is Continuous Integration (CI) and why is it important in DevOps practices?
- 2) How does CI differ from Continuous Deployment (CD) in the software development lifecycle?
- 3) What are the key benefits of adopting CI in a software development team?
- 4) How does CI help in detecting software issues earlier in the development cycle?
- 5) How can CI processes help in avoiding code conflicts between developers?
- 6) What is a "CI Pipeline" and how does it work in DevOps practices?
- 7) How do teams manage changes in continuously integrated source code?
- 8) What are the common challenges faced in adopting CI in software development, and how to overcome them?

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

